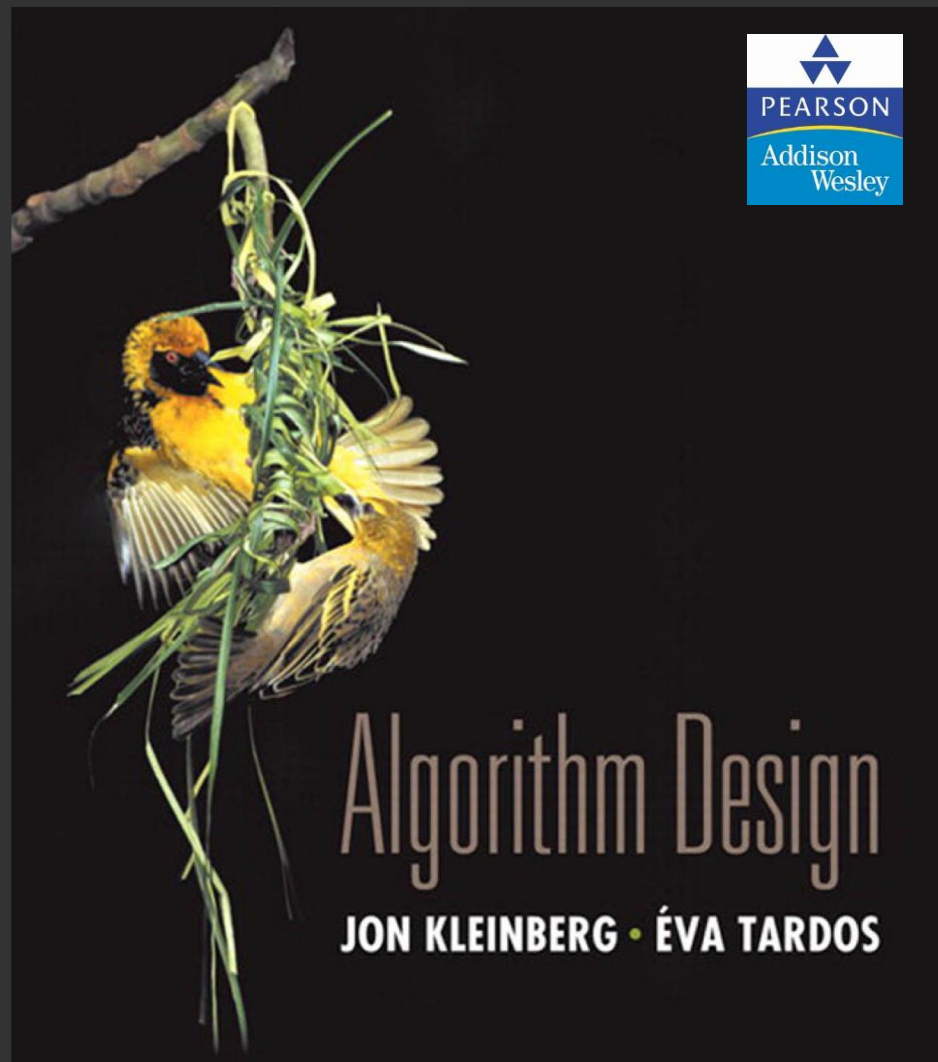




8. INTRACTABILITY II

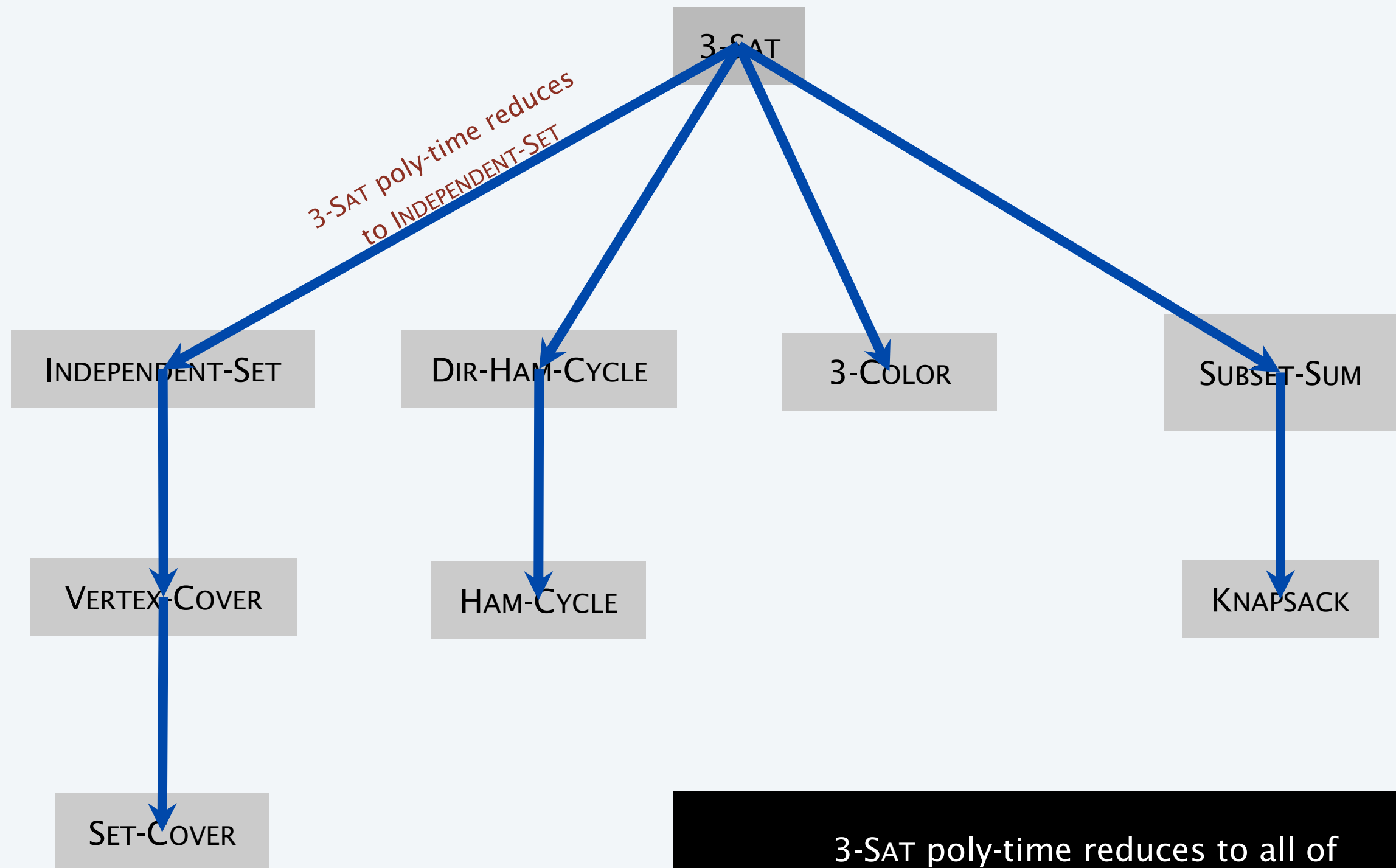
- P vs. NP
- NP -complete
- co - NP

Lecture slides by Kevin Wayne

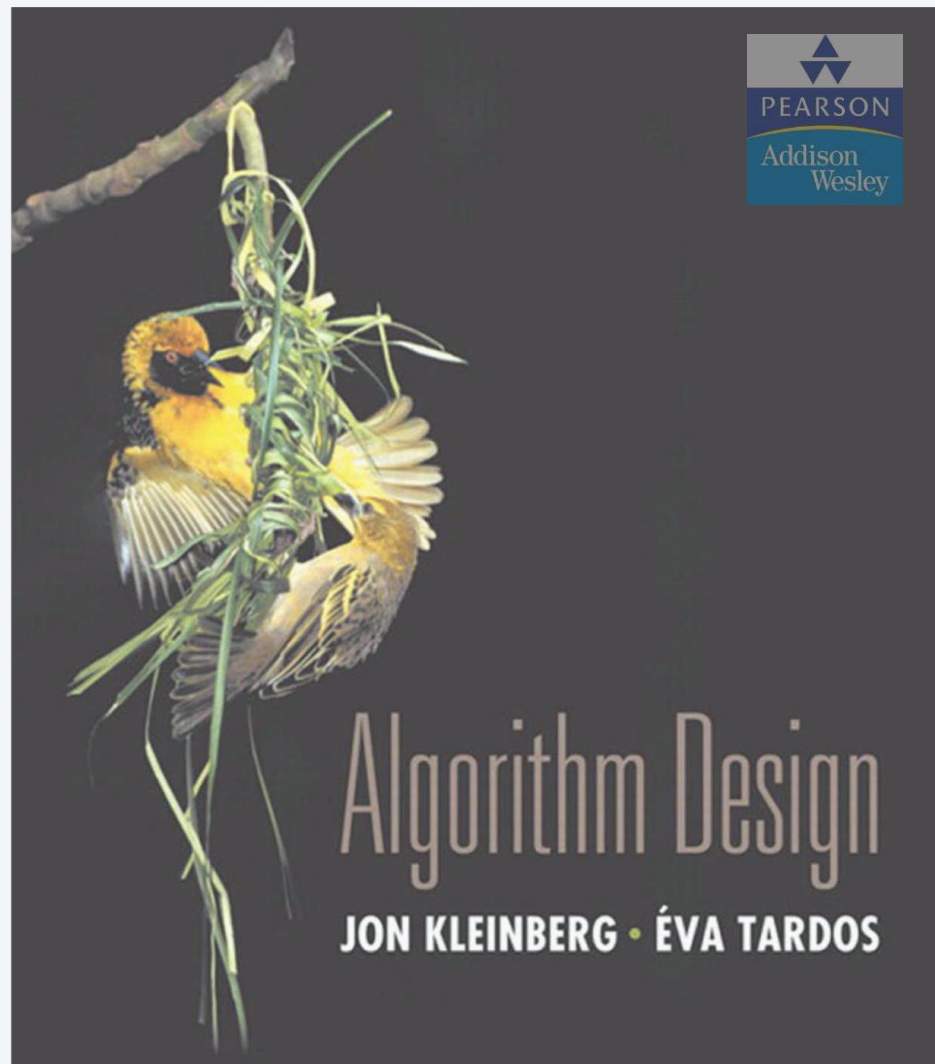
Copyright © 2005 Pearson-Addison Wesley

<http://www.cs.princeton.edu/~wayne/kleinberg-tardos>

Recap



3-SAT poly-time reduces to all of these problems (and many, many more)



SECTION 8.3

8. INTRACTABILITY II

- P vs. NP
- NP -complete
- co - NP

Decision problem.

- Problem X is a set of strings.
- Instance s is one string.
- Algorithm A solves problem X : $A(s) = \begin{cases} \text{yes} & \text{if } s \in X \\ \text{no} & \text{if } s \notin X \end{cases}$

Def. Algorithm A runs in **polynomial time** if for every string s , $A(s)$ terminates in $\leq p(|s|)$ “steps,” where $p(\cdot)$ is some polynomial function.

↑
length of s

Def. $\mathbf{P}$ = set of decision problems for which there exists a poly-time algorithm.

↑
on a deterministic
Turing machine

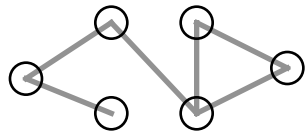
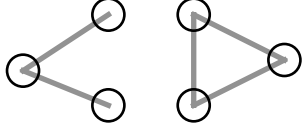
problem PRIMES: { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, ... }

instance s : 592335744548702854681

algorithm: Agrawal–Kayal–Saxena (2002)

Some problems in P

P. Decision problems for which there exists a poly-time algorithm.

problem	description	poly-time algorithm	yes	no
MULTIPLE	Is x a multiple of y ?	grade-school division	51, 17	51, 16
REL-PRIME	Are x and y relatively prime ?	Euclid's algorithm	34, 39	34, 51
PRIMES	Is x prime ?	Agrawal-Kayal-Saxena	53	51
EDIT-DISTANCE	Is the edit distance between x and y less than 5 ?	Needleman-Wunsch	niether neither	acgggt tttta
L-SOLVE	Is there a vector x that satisfies $Ax = b$?	Gauss-Edmonds elimination	$\begin{bmatrix} 0 & 1 & 1 \\ 2 & 4 & -2 \\ 0 & 3 & 15 \end{bmatrix}, \begin{bmatrix} 4 \\ 2 \\ 36 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$
U-CONN	Is an undirected graph G connected?	depth-first search		

Def. Algorithm $C(s, t)$ is a **certifier** for problem X if for every string s :
 $s \in X$ iff there exists a string t such that $C(s, t) = \text{yes}$.

← TUTTI QUEI PROBLEMI DI DECISIONE PER CUI ESISTE UN CERTIFICATORE POLINOMIALE

Def. NP = set of decision problems for which there exists a poly-time certifier.

- $C(s, t)$ is a poly-time algorithm.
- Certificate t is of polynomial size: $|t| \leq p(|s|)$ for some polynomial $p(\cdot)$.

↑
 “certificate” or “witness”

problem COMPOSITES:	$\{ 4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, \dots \}$
instance s:	437669
certificate t:	541 ← $437,669 = 541 \times 809$
certifier $C(s, t)$:	grade-school division

Certifiers and certificates: satisfiability

SAT. Given a CNF formula Φ , does it have a satisfying truth assignment?

3-SAT. SAT where each clause contains exactly 3 literals.

Certificate. An assignment of truth values to the Boolean variables.

Certifier. Check that each clause in Φ has at least one true literal.

instance s $\Phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

certificate t $x_1 = \text{true}, x_2 = \text{true}, x_3 = \text{false}, x_4 = \text{false}$

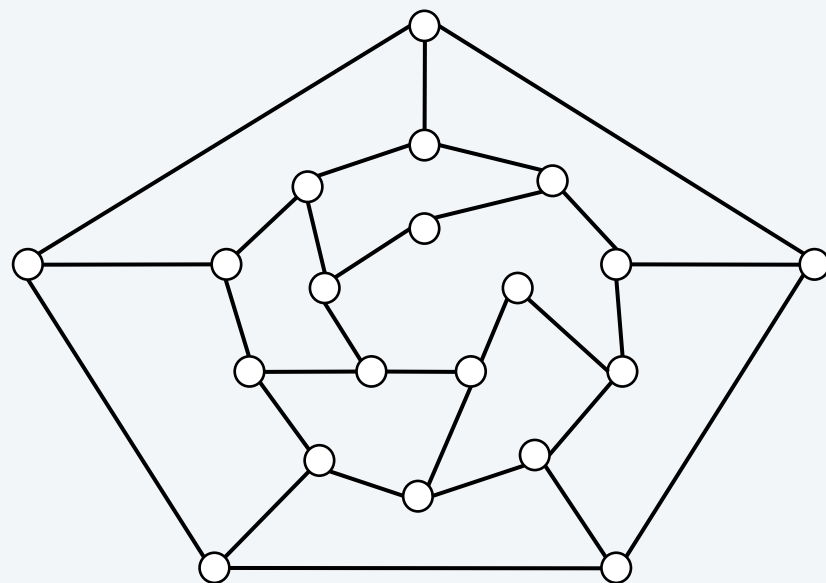
Conclusions. SAT $\in$ **NP**, 3-SAT $\in$ **NP**.

Certifiers and certificates: Hamilton path

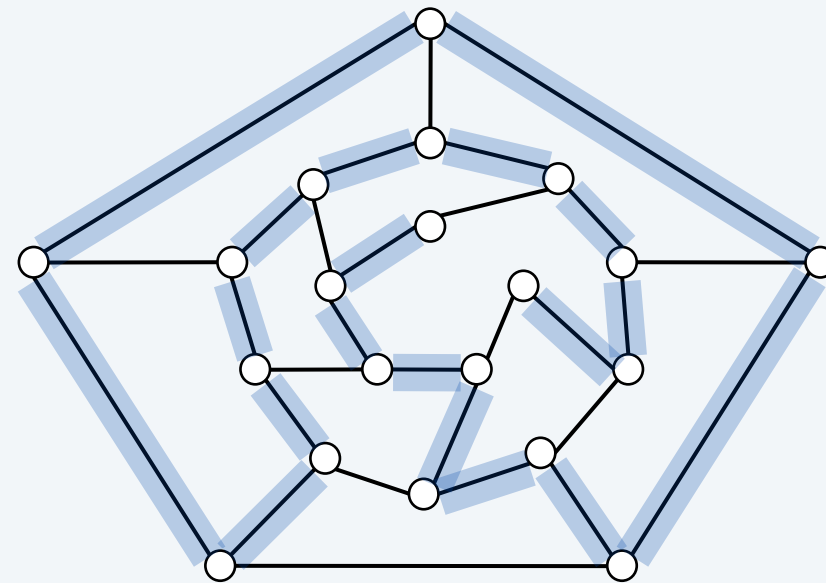
HAMILTON-PATH. Given an undirected graph $G = (V, E)$, does there exist a simple path P that visits every node?

Certificate. A permutation π of the n nodes.

Certifier. Check that π contains each node in V exactly once, and that G contains an edge between each pair of adjacent nodes.



instance s



certificate t

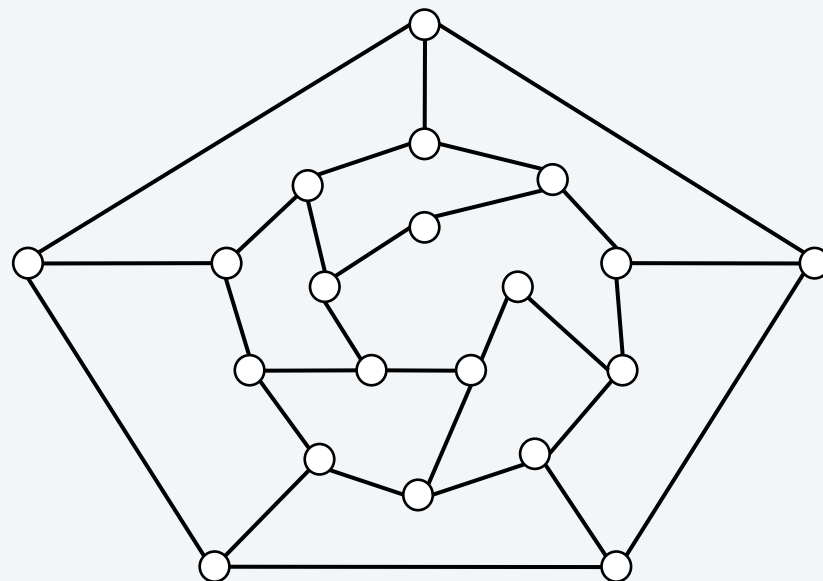
Conclusion. HAMILTON-PATH $\in$ **NP**.

Two problems that probably do not belong to NP

CHECKERS. Given a board position in an n -by- n generalization of checkers, can black guarantee a win?

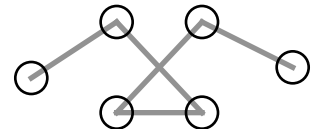
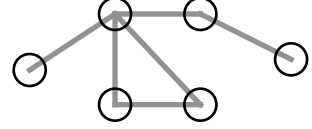


CO-LONGEST-PATH. Given an undirected graph $G = (V, E)$, is the length of the longest simple path $\leq k$?



Some problems in NP

NP. Decision problems for which there exists a poly-time certifier.

problem	description	poly-time algorithm	yes	no
L-SOLVE	Is there a vector x that satisfies $Ax = b$?	Gauss-Edmonds elimination	$\begin{bmatrix} 0 & 1 & 1 \\ 2 & 4 & -2 \\ 0 & 3 & 15 \end{bmatrix}, \begin{bmatrix} 4 \\ 2 \\ 36 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$
COMPOSITES	Is x composite?	Agrawal-Kayal-Saxena	51	53
FACTOR	Does x have a nontrivial factor less than y ?	???	(56159, 50)	(55687, 50)
SAT	Given a CNF formula, does it have a satisfying truth assignment?	???	$\neg x_1 \vee x_2 \vee \neg x_3$ $x_1 \vee \neg x_2 \vee x_3$ $\neg x_1 \vee \neg x_2 \vee x_3$	$\neg x_2$ $x_1 \vee x_2$ $\neg x_1 \vee x_2$
HAMILTON-PATH	Is there a simple path between u and v that visits every node?	???		

P, NP, and EXP

P. Decision problems for which there exists a poly-time algorithm.

NP. Decision problems for which there exists a poly-time certifier.

EXP. Decision problems for which there exists an exponential-time algorithm.

Proposition. $\mathbf{P} \subseteq \mathbf{NP}$.

Pf. Consider any problem $X \in \mathbf{P}$.

- By definition, there exists a poly-time algorithm $A(s)$ that solves X .
- Certificate $t = \varepsilon$, certifier $C(s, t) = A(s)$. ▪

Proposition. $\mathbf{NP} \subseteq \mathbf{EXP}$.

Pf. Consider any problem $X \in \mathbf{NP}$.

- By definition, there exists a poly-time certifier $C(s, t)$ for X , where certificate t satisfies $|t| \leq p(|s|)$ for some polynomial $p(\cdot)$.
- To solve instance s , run $C(s, t)$ on all strings t with $|t| \leq p(|s|)$.
- Return *yes* iff $C(s, t)$ returns *yes* for any of these potential certificates. ▪

UNICA
SLIDE
DA
FARE

Fact. $\mathbf{P} \neq \mathbf{EXP} \Rightarrow$ either $\mathbf{P} \neq \mathbf{NP}$, or $\mathbf{NP} \neq \mathbf{EXP}$, or both.

The main question: P vs. NP

$P=NP?$ SI 👍

Q. How to solve an instance of 3-SAT with n variables?

A. Exhaustive search: try all 2^n truth assignments.

Q. Can we do anything substantially more clever?

Conjecture. No poly-time algorithm for 3-SAT.

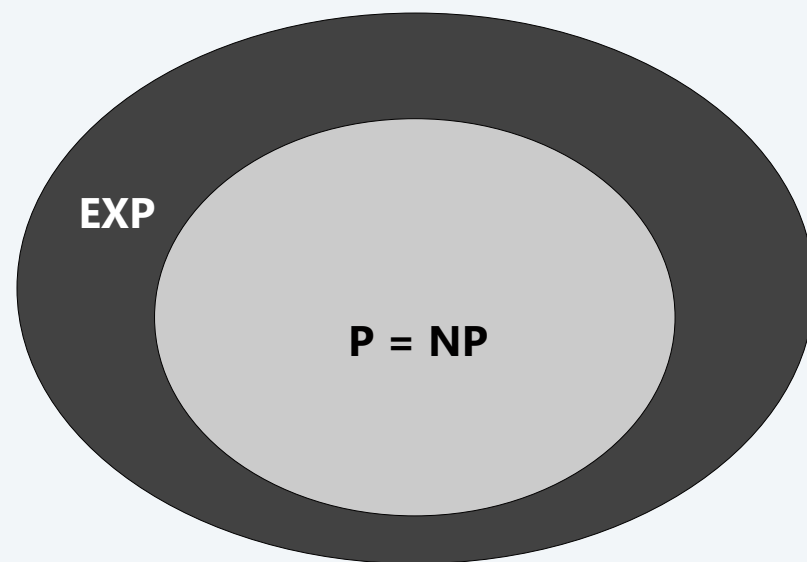
“intractable”



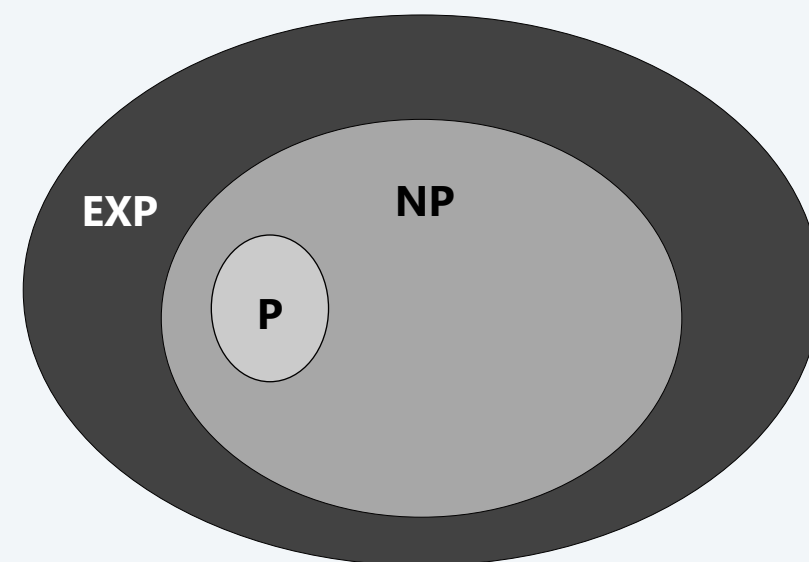
The main question: P vs. NP

Does $P = NP$? [Cook 1971, Edmonds, Levin, Yablonski, Gödel]

Is the decision problem as easy as the certification problem?



If $P = NP$



If $P \neq NP$

If yes... Efficient algorithms for 3-SAT, TSP, VERTEX-COVER, FACTOR, ...

If no... No efficient algorithms possible for 3-SAT, TSP, VERTEX-COVER, ...

Consensus opinion. Probably no.

Possible outcomes

P $\neq$ NP

*“ I conjecture that there is no good algorithm for the traveling salesman problem. My reasons are the same as for any mathematical conjecture:
(i) It is a legitimate mathematical possibility and (ii) I do not know. ”*

— *Jack Edmonds 1966*



“ In my view, there is no way to even make intelligent guesses about the answer to any of these questions. If I had to bet now, I would bet that P is not equal to NP. I estimate the half-life of this problem at 25–50 more years, but I wouldn’t bet on it being solved before 2100. ”

— *Bob Tarjan (2002)*



Possible outcomes

P = NP

“ I think that in this respect I am on the loony fringe of the mathematical community: I think (not too strongly!) that $P=NP$ and this will be proved within twenty years. Some years ago, Charles Read and I worked on it quite bit, and we even had a celebratory dinner in a good restaurant before we found an absolutely fatal mistake. ”

— Béla Bollobás (2002)



“ In my opinion this shouldn't really be a hard problem; it's just that we came late to this theory, and haven't yet developed any techniques for proving computations to be hard. Eventually, it will just be a footnote in the books. ” — John Conway



Other possible outcomes

P = NP, but only $\Omega(n^{100})$ algorithm for 3-SAT.

P $\neq$ NP, but with $O(n^{\log^* n})$ algorithm for 3-SAT.

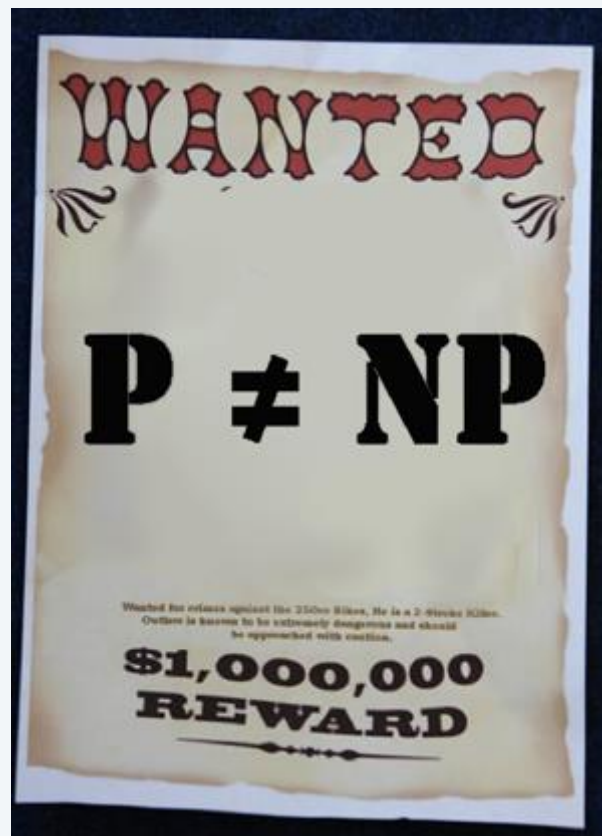
P = NP is independent (of ZFC axiomatic set theory).

“ It will be solved by either 2048 or 4096. I am currently somewhat pessimistic. The outcome will be the truly worst case scenario: namely that someone will prove $P = NP$ because there are only finitely many obstructions to the opposite hypothesis; hence there exists a polynomial time solution to SAT but we will never know its complexity! ” — Donald Knuth



Millennium prize

Millennium prize. \$1 million for resolution of $P \neq NP$ problem.



Clay Mathematics Institute
Dedicated to increasing and disseminating mathematical knowledge

HOME | ABOUT CMI | PROGRAMS | NEWS & EVENTS | AWARDS | SCHOLARS | PUBLICATIONS

Millennium Problems

In order to celebrate mathematics in the new millennium, The Clay Mathematics Institute of Cambridge, Massachusetts (CMI) has named seven *Prize Problems*. The Scientific Advisory Board of CMI selected these problems, focusing on important classic questions that have resisted solution over the years. The Board of Directors of CMI designated a \$7 million prize fund for the solution to these problems, with \$1 million allocated to each. During the [Millennium Meeting](#) held on May 24, 2000 at the Collège de France, Timothy Gowers presented a lecture entitled *The Importance of Mathematics*, aimed for the general public, while John Tate and Michael Atiyah spoke on the problems. The CMI invited specialists to formulate each problem.

- ▶ [Birch and Swinnerton-Dyer Conjecture](#)
- ▶ [Hodge Conjecture](#)
- ▶ [Navier-Stokes Equations](#)
- ▶ [P vs NP](#)
- ▶ [Poincaré Conjecture](#)
- ▶ [Riemann Hypothesis](#)
- ▶ [Yang-Mills Theory](#)
- ▶ [Rules](#)
- ▶ [Millennium Meeting Videos](#)

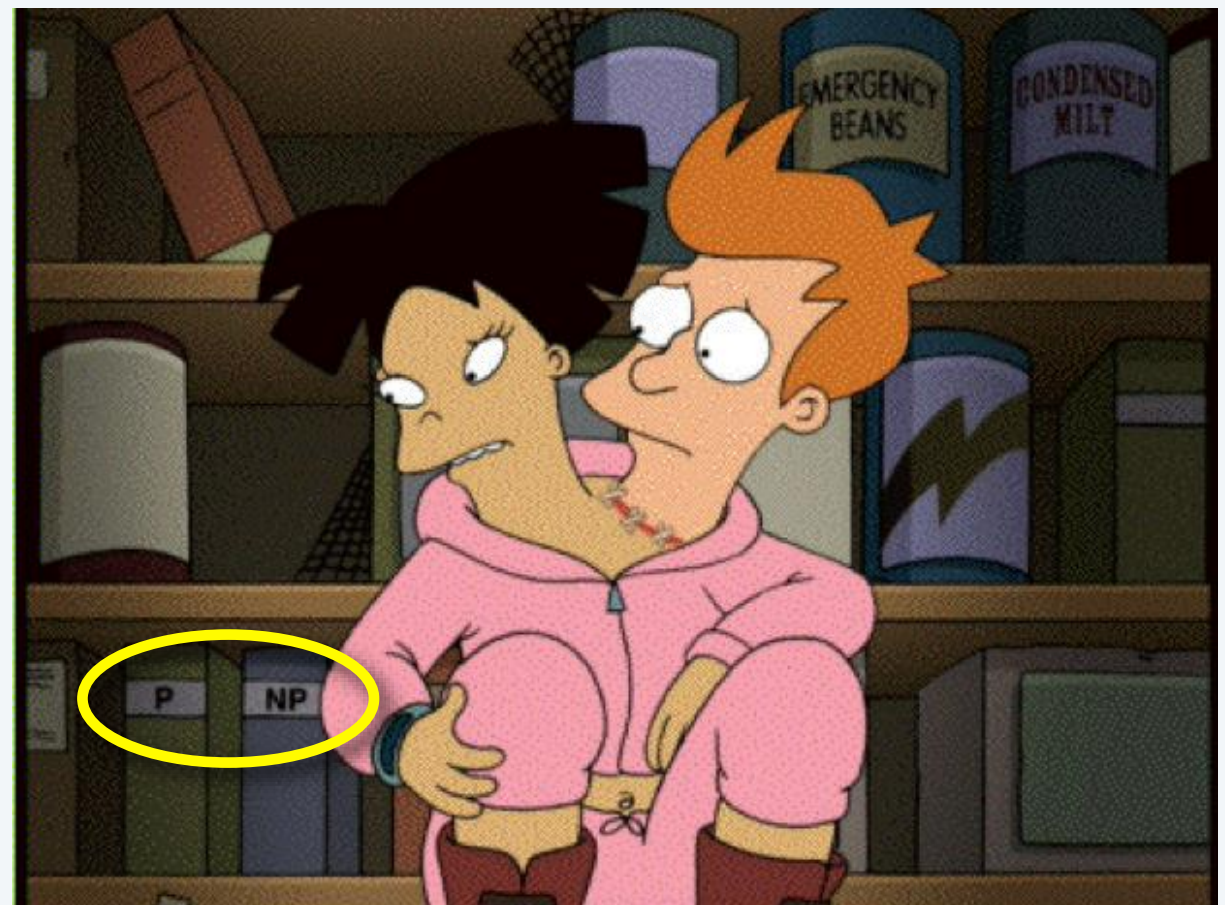
P vs. NP and pop culture

Some writers for the Simpsons and Futurama.

- J. Stewart Burns. *M.S. in mathematics (Berkeley '93).*
- David X. Cohen. *M.S. in computer science (Berkeley '92).*
- Al Jean. *B.S. in mathematics. (Harvard '81).*
- Ken Keeler. *Ph.D. in applied mathematics (Harvard '90).*
- Jeff Westbrook. *Ph.D. in computer science (Princeton '89).*



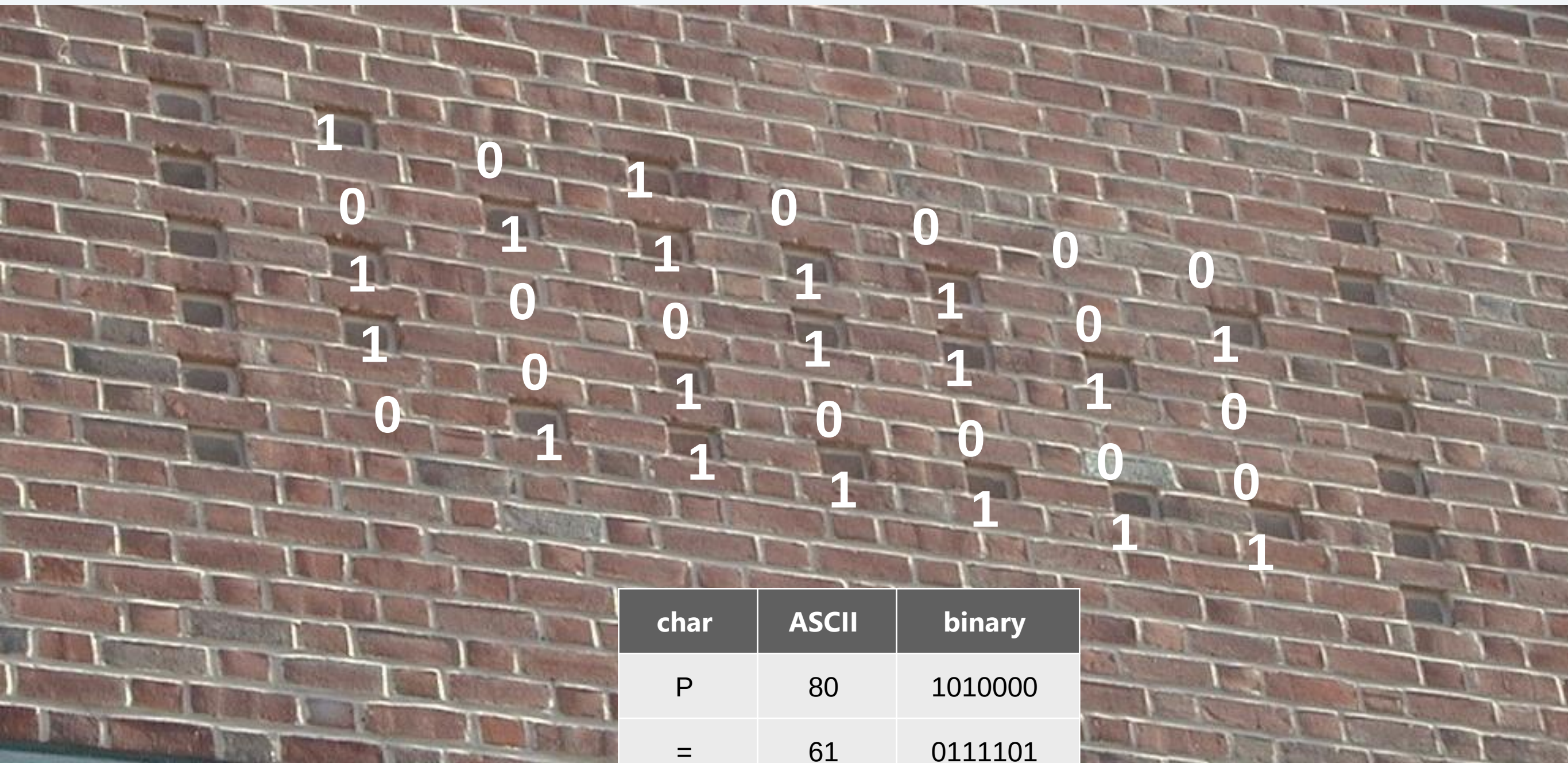
Copyright © 1990, Matt Groening



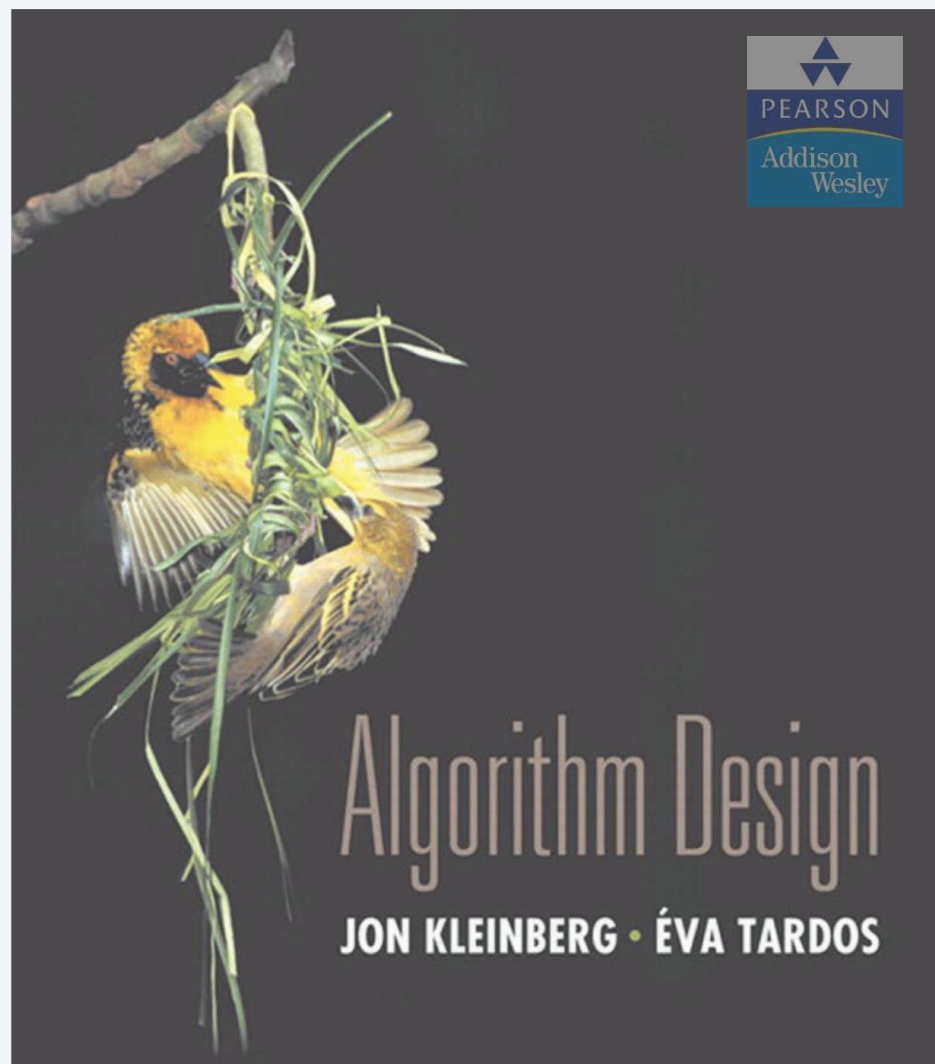
Copyright © 2000, Twentieth Century Fox



Princeton CS Building, West Wall, Circa 2001



char	ASCII	binary
P	80	1010000
=	61	0111101
N	78	1001110
P	80	1010000
?	63	0111111



SECTION 8.4

8. INTRACTABILITY II

- *P vs. NP*
- *NP-complete*
- *co-NP*

1. Polynomial Reduction (riduzione polinomiale)

- È il concetto generale:
Un problema X si **riduce** a un problema Y se esiste un algoritmo che **trasforma ogni istanza di X in un'istanza di Y in tempo polinomiale**, tale che la risposta ("yes/no") resti la stessa.
- Notazione:

$$X \leq_p Y$$

- Esempio:
 - $\text{SAT} \leq_p \text{3-SAT}$ (puoi trasformare una formula SAT in una formula 3-SAT in tempo polinomiale).

👉 Le polynomial reductions servono per confrontare la **difficoltà dei problemi** e definire classi come NP-completezza.

2. Polynomial Transformation (trasformazione polinomiale)

- È il meccanismo tecnico che realizza la riduzione.
- È la **funzione concreta** f che prende un'istanza s di X e costruisce un'istanza $f(s)$ di Y .
- Requisiti:
 - f calcolabile in tempo polinomiale.
 - s è istanza "yes" di $X \Leftrightarrow f(s)$ è istanza "yes" di Y .

👉 In pratica, la *polynomial transformation* è l'atto operativo, la *polynomial reduction* è il concetto più generale che dice "X è almeno difficile quanto Y".

Polynomial transformations

Def. Problem X **polynomial (Cook) reduces** to problem Y if arbitrary instances of problem X can be solved using:

- Polynomial number of standard computational steps, plus
- Polynomial number of calls to oracle that solves problem Y .

Def. Problem X **polynomial (Karp) transforms** to problem Y if given any instance x of X , we can construct an instance y of Y such that x is a *yes* instance of X iff y is a *yes* instance of Y .

↑
we require $|y|$ to be of size polynomial in $|x|$

Note. Polynomial transformation is polynomial reduction with just one call to oracle for Y , exactly at the end of the algorithm for X . Almost all previous reductions were of this form.

Open question. Are these two concepts the same with respect to **NP**?

↑
we abuse notation $\leq_p$ and blur distinction

NP-complete

NP-complete. A problem $Y \in \mathbf{NP}$ with the property that for every problem $X \in \mathbf{NP}$, $X \leq_p Y$.

Proposition. Suppose $Y \in \mathbf{NP}$ -complete. Then, $Y \in \mathbf{P}$ iff $\mathbf{P} = \mathbf{NP}$.

Pf. $\Leftarrow$ If $\mathbf{P} = \mathbf{NP}$, then $Y \in \mathbf{P}$ because $Y \in \mathbf{NP}$.

Pf. $\Rightarrow$ Suppose $Y \in \mathbf{P}$.

- Consider any problem $X \in \mathbf{NP}$. Since $X \leq_p Y$, we have $X \in \mathbf{P}$.
- This implies $\mathbf{NP} \subseteq \mathbf{P}$.
- We already know $\mathbf{P} \subseteq \mathbf{NP}$. Thus $\mathbf{P} = \mathbf{NP}$. ▪

Fundamental question. Are there any “natural” **NP**-complete problems?

The "first" NP-complete problem

Theorem. [Cook 1971, Levin 1973] SAT ∈ NP-complete.

The Complexity of Theorem-Proving Procedures

Stephen A. Cook

University of Toronto

Summary

It is shown that any recognition problem solved by a polynomial time-bounded nondeterministic Turing machine can be "reduced" to the problem of determining whether a given propositional formula is a tautology. Here "reduced" means, roughly speaking, that the first problem can be solved deterministically in polynomial time provided an oracle is available for solving the second. From this notion of reducible, polynomial degrees of difficulty are defined, and it is shown that the problem of determining tautologyhood has the same polynomial degree as the problem of determining whether the first of two given graphs is isomorphic to a subgraph of the second. Other examples are discussed. A method of measuring the complexity of proof procedures for the predicate calculus is introduced and discussed.

Throughout this paper, a set of strings means a set of strings on some fixed, large, finite alphabet Σ . This alphabet is large enough to include symbols for all sets described here. All Turing machines are deterministic recognition devices, unless the contrary is explicitly stated.

1. Tautologies and Polynomial Reducibility.

Let us fix a formalism for the propositional calculus in which formulas are written as strings on Σ . Since we will require infinitely many proposition symbols (atoms), each such symbol will consist of a member of Σ followed by a number in binary notation to distinguish that symbol. Thus a formula of length n can only have about $n/\log n$ distinct function and predicate symbols. The logical connectives are $\&$ (and), $\vee$ (or), and $\neg$ (not).

The set of tautologies (denoted by {tautologies}) is a

certain recursive set of strings on this alphabet, and we are interested in the problem of finding a good lower bound on its possible recognition times. We provide no such lower bound here, but theorem 1 will give evidence that {tautologies} is a difficult set to recognize, since many apparently difficult problems can be reduced to determining tautologyhood. By reduced we mean, roughly speaking, that if tautologyhood could be decided instantly (by an "oracle") then these problems could be decided in polynomial time. In order to make this notion precise, we introduce query machines, which are like Turing machines with oracles in [1].

A query machine is a multitape Turing machine with a distinguished tape called the query tape, and three distinguished states called the query state, yes state, and no state, respectively. If M is a query machine and T is a set of strings, then a T-computation of M is a computation of M in which initially M is in the initial state and has an input string w on its input tape, and each time M assumes the query state there is a string u on the query tape, and the next state M assumes is the yes state if $u \in T$ and the no state if $u \notin T$. We think of an "oracle", which knows T , placing M in the yes state or no state.

Definition

A set S of strings is P-reducible (P for polynomial) to a set T of strings iff there is some query machine M and a polynomial $Q(n)$ such that for each input string w , the T-computation of M with input w halts within $Q(|w|)$ steps ($|w|$ is the length of w), and ends in an accepting state iff $w \in S$.

It is not hard to see that P-reducibility is a transitive relation. Thus the relation E on

ПРОБЛЕМЫ ПЕРЕДАЧИ ИНФОРМАЦИИ

Том IX

1973

Вып. 3

КРАТКИЕ СООБЩЕНИЯ

УДК 519.14

УНИВЕРСАЛЬНЫЕ ЗАДАЧИ ПЕРЕБОРА

Л. А. Левин

В статье рассматриваются несколько известных массовых задач «переборного типа» и доказывается, что эти задачи можно решать лишь за такое время, за которое можно решать вообще любые задачи указанного типа.

После уточнения понятия алгоритма была доказана алгоритмическая неразрешимость ряда классических массовых проблем (например, проблем тождества элементов групп, гомеоморфности многообразий, разрешимости диофантовых уравнений и других). Тем самым был снят вопрос о нахождении практического способа их решения. Однако существование алгоритмов для решения других задач не снимает для них аналогичного вопроса из-за фантастически большого объема работы, предсказываемого этими алгоритмами. Такова ситуация с так называемыми переборными задачами: минимизации булевых функций, поиска доказательств ограниченной длины, выяснения изоморфности графов и другими. Все эти задачи решаются тривиальными алгоритмами, состоящими в переборе всех возможностей. Однако эти алгоритмы требуют экспоненциального времени работы и у математиков сложилось убеждение, что более простые алгоритмы для них невозможны. Был получен ряд серьезных аргументов в пользу его справедливости (см. [1, 2]), однако доказать это утверждение не удалось никому. (Например, до сих пор не доказано, что для нахождения математических доказательств нужно больше времени, чем для их проверки.)

Однако если предположить, что вообще существует какая-нибудь (хотя бы искусственно построенная) массовая задача переборного типа, неразрешимая простыми (в смысле объема вычислений) алгоритмами, то можно показать, что этим же свойством обладают и многие «классические» переборные задачи (в том числе задача минимизации, задача поиска доказательств и др.). В этом и состоят основные результаты статьи.

Функции $f(n)$ и $g(n)$ будем называть сравнимыми, если при некотором k

$$f(n) \leq (g(n) + 2)^k \quad \text{и} \quad g(n) \leq (f(n) + 2)^k.$$

Аналогично будем понимать термин «меньше или сравнимо».

О п р е д е л е н и е. Задачей переборного типа (или просто переборной задачей) будем называть задачу вида «по данному x найти какое-нибудь y длины, сравнимой с длиной x , такое, что выполняется $A(x, y)$ », где $A(x, y)$ — какое-нибудь свойство, проверяемое алгоритмом, время работы которого сравнимо с длиной x . (Под алгоритмом здесь можно понимать, например, алгоритмы Колмогорова — Успенского или машины Тьюринга, или нормальные алгоритмы; x, y — двоичные слова). Квазипереборной задачей будем называть задачу выяснения, существует ли такое y .

Мы рассмотрим шесть задач этих типов. Рассматриваемые в них объекты кодируются естественным образом в виде двоичных слов. При этом выбор естественной кодировки не существен, так как все они дают сравнимые длины кодов.

Задача 1. Заданы списоком конечное множество и покрытие его 500-элементными подмножествами. Найти подпокрытие заданной мощности (соответственно выяснить существует ли оно).

Задача 2. Таблично задана частичная булева функция. Найти заданного размера дизъюнктивную нормальную форму, реализующую эту функцию в области определения (соответственно выяснить существует ли она).

Задача 3. Выяснить, выводима или опровержима данная формула исчисления высказываний. (Или, что то же самое, равна ли константе данная булева формула.)

Задача 4. Даны два графа. Найти гомоморфизм одного на другой (выяснить его существование).

Задача 5. Даны два графа. Найти изоморфизм одного в другой (на его часть).

Задача 6. Рассматриваются матрицы из целых чисел от 1 до 100 и некоторое условие о том, какие числа в них могут соседствовать по вертикали и какие по горизонтали. Заданы числа на границе и требуется продолжить их на всю матрицу с соблюдением условия.

Establishing NP-completeness

Remark. Once we establish first “natural” **NP**-complete problem, others fall like dominoes.

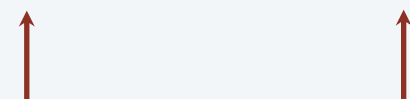
Recipe. To prove that $Y \in \mathbf{NP}$ -complete:

- Step 1. Show that $Y \in \mathbf{NP}$.
- Step 2. Choose an **NP**-complete problem X .
- Step 3. Prove that $X \leq_p Y$.

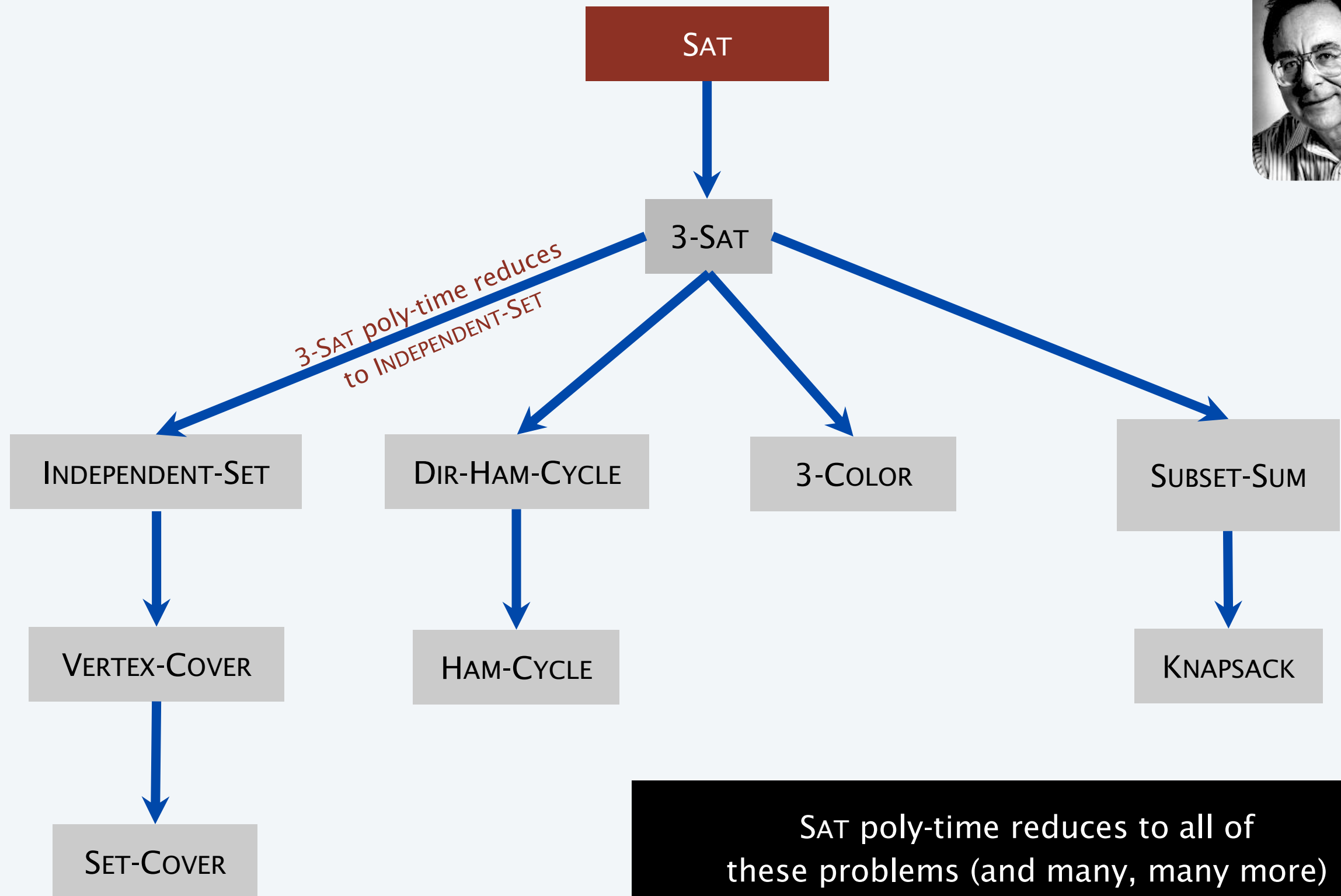
Proposition. If $X \in \mathbf{NP}$ -complete, $Y \in \mathbf{NP}$, and $X \leq_p Y$, then $Y \in \mathbf{NP}$ -complete.

Pf. Consider any problem $W \in \mathbf{NP}$. Then, both $W \leq_p X$ and $X \leq_p Y$.

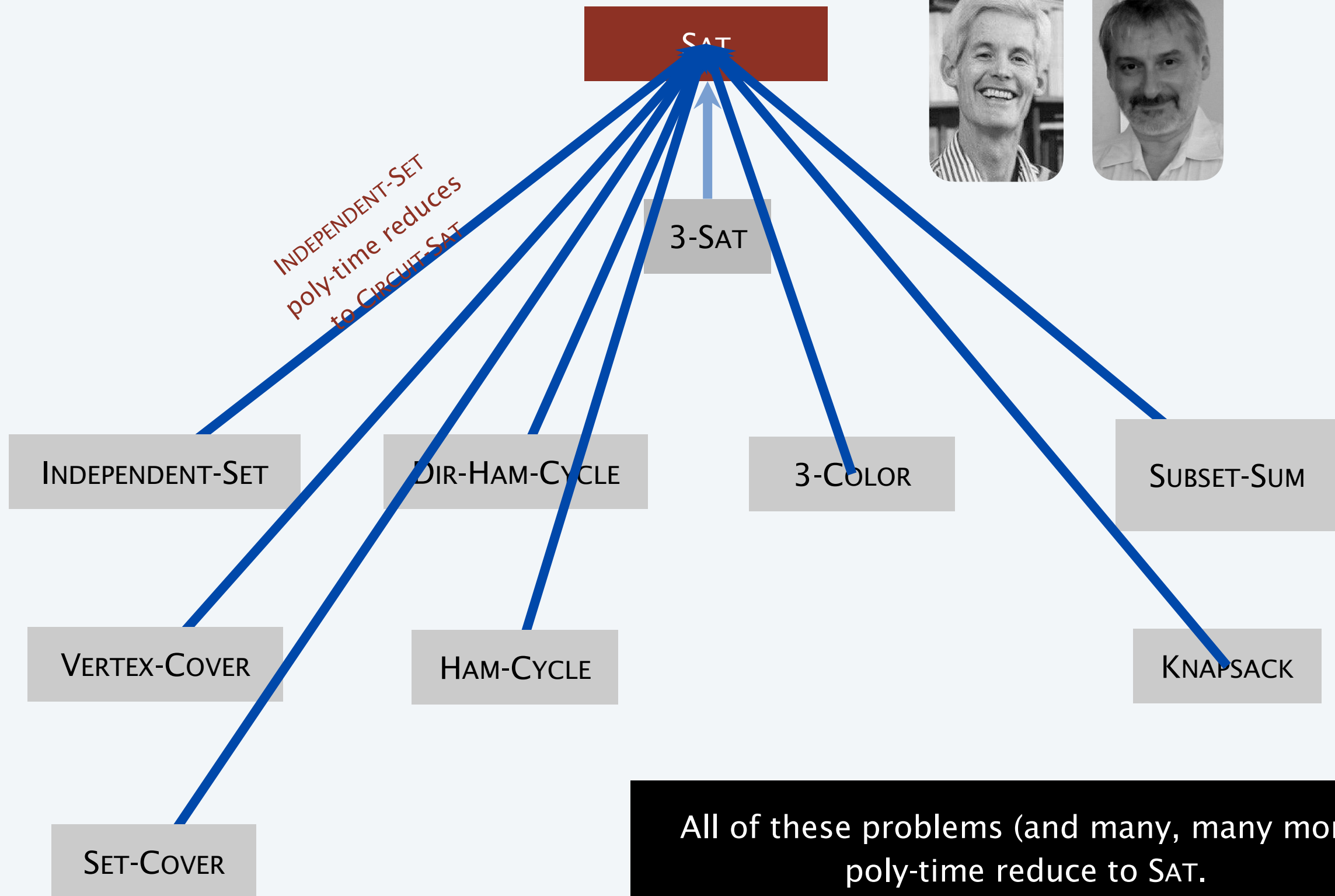
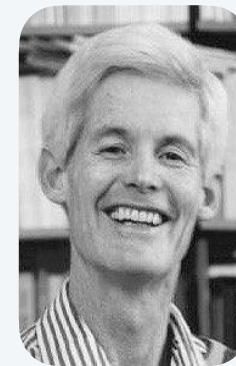
- By transitivity, $W \leq_p Y$.
- Hence $Y \in \mathbf{NP}$ -complete. ▪


by definition of **NP**-complete by assumption

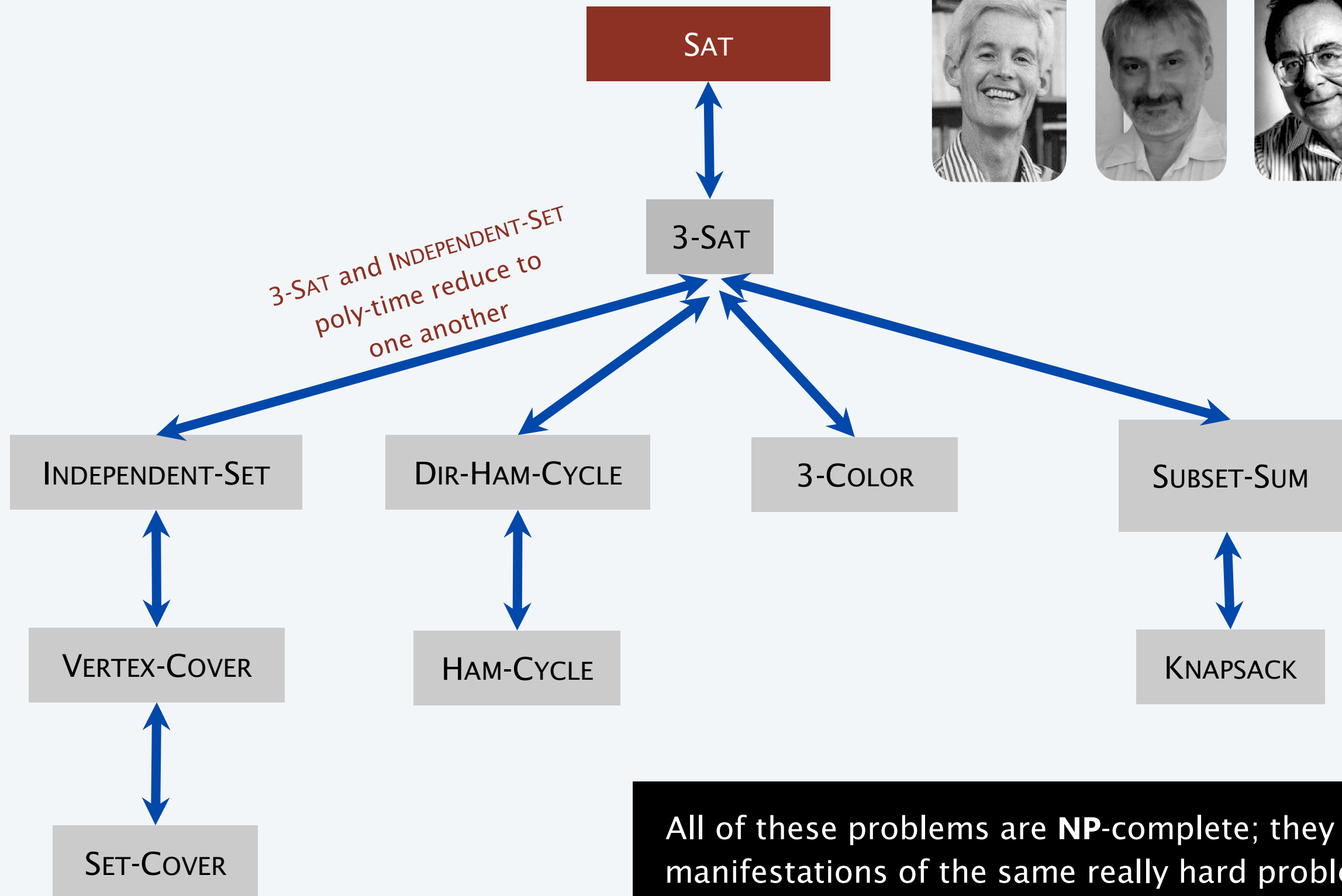
Implications of Karp



Implications of Cook–Levin



Implications of Karp + Cook-Levin



I'D TELL YOU ANOTHER
NP-COMPLETE JOKE,
BUT ONCE YOU'VE HEARD ONE,

YOU'VE HEARD THEM
ALL.

Some NP-complete problems

Basic genres of NP-complete problems and paradigmatic examples.

- Packing/covering problems: SET-COVER, VERTEX-COVER, INDEPENDENT-SET.
- Constraint satisfaction problems: CIRCUIT-SAT, SAT, 3-SAT.
- Sequencing problems: HAMILTON-CYCLE, TSP.
- Partitioning problems: 3D-MATCHING, 3-COLOR.
- Numerical problems: SUBSET-SUM, KNAPSACK.

Practice. Most **NP** problems are known to be either in **P** or **NP**-complete.

NP-intermediate? FACTOR, DISCRETE-LOG, GRAPH-ISOMORPHISM,

Theorem. [Ladner 1975] Unless **P** = **NP**, there exist problems in **NP** that are neither in **P** nor **NP**-complete.

On the Structure of Polynomial Time Reducibility

RICHARD E. LADNER

University of Washington, Seattle, Washington

More hard computational problems

Garey and Johnson. Computers and Intractability.

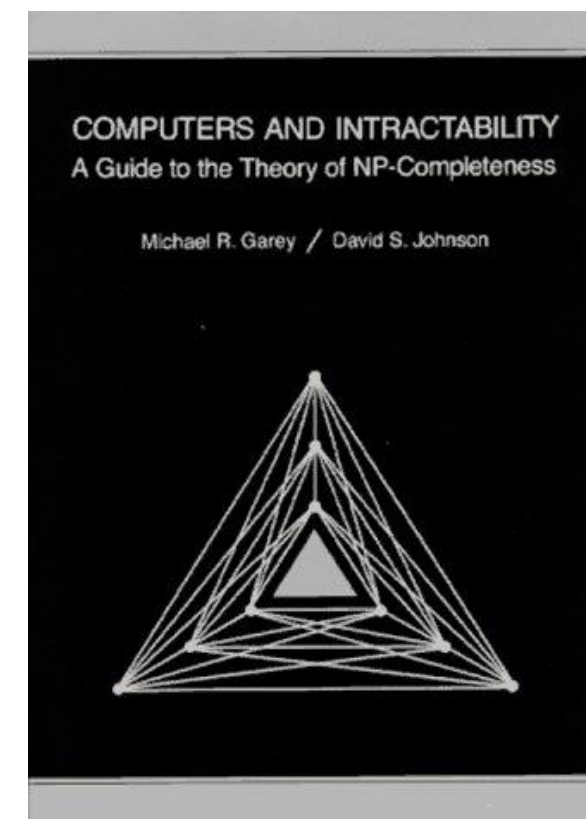
- Appendix includes over 300 **NP**-complete problems.
- Most cited reference in computer science literature.

Most Cited Computer Science Citations

This list is generated from documents in the CiteSeer^x database as of January 17, 2013. This list is automatically generated and may contain errors. The list is generated in batch mode and citation counts may differ from those currently in the CiteSeer^x database, since the database is continuously updated.

[All Years](#) | [1990](#) | [1991](#) | [1992](#) | [1993](#) | [1994](#) | [1995](#) | [1996](#) | [1997](#) | [1998](#) | [1999](#) | [2000](#) | [2001](#) | [2002](#) | [2003](#) | [2004](#) | [2005](#) | [2006](#) | [2007](#) | [2008](#) | [2009](#) | [2010](#) | [2011](#) | [2012](#) | [2013](#)

1. M R Garey, D S Johnson
[Computers and Intractability. A Guide to the Theory of NP-Completeness](#) 1979
8665
2. T Cormen, C E Leiserson, R Rivest
[Introduction to Algorithms](#) 1990
7210
3. V N Vapnik
[The nature of statistical learning theory](#) 1998
6580
4. A P Dempster, N M Laird, D B Rubin
[Maximum likelihood from incomplete data via the EM algorithm](#). Journal of the Royal Statistical Society, 1977
6082
5. T Cover, J Thomas
[Elements of Information Theory](#) 1991
6075
6. D E Goldberg
[Genetic Algorithms](#) in Search, Optimization, and Machine Learning, 1989
5998
7. J Pearl
[Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference](#) 1988
5582
8. E Gamma, R Helm, R Johnson, J Vlissides
[Design Patterns: Elements of Reusable Object-Oriented Software](#) 1995
4614
9. C E Shannon
[A mathematical theory of communication](#) Bell Syst. Tech. J, 1948
4118
10. J R Quinlan
[C4.5: Programs for Machine Learning](#) 1993
4018



More hard computational problems

Aerospace engineering. Optimal mesh partitioning for finite elements.

Biology. Phylogeny reconstruction.

Chemical engineering. Heat exchanger network synthesis.

Chemistry. Protein folding.

Civil engineering. Equilibrium of urban traffic flow.

Economics. Computation of arbitrage in financial markets with friction.

Electrical engineering. VLSI layout.

Environmental engineering. Optimal placement of contaminant sensors.

Financial engineering. Minimum risk portfolio of given return.

Game theory. Nash equilibrium that maximizes social welfare.

Mathematics. Given integer $a_1, \dots, a_n$, compute $\int_0^{2\pi} \cos(a_1\theta) \times \cos(a_2\theta) \times \dots \times \cos(a_n\theta) d\theta$

Mechanical engineering. Structure of turbulence in sheared flows.

Medicine. Reconstructing 3d shape from biplane angiocardigram.

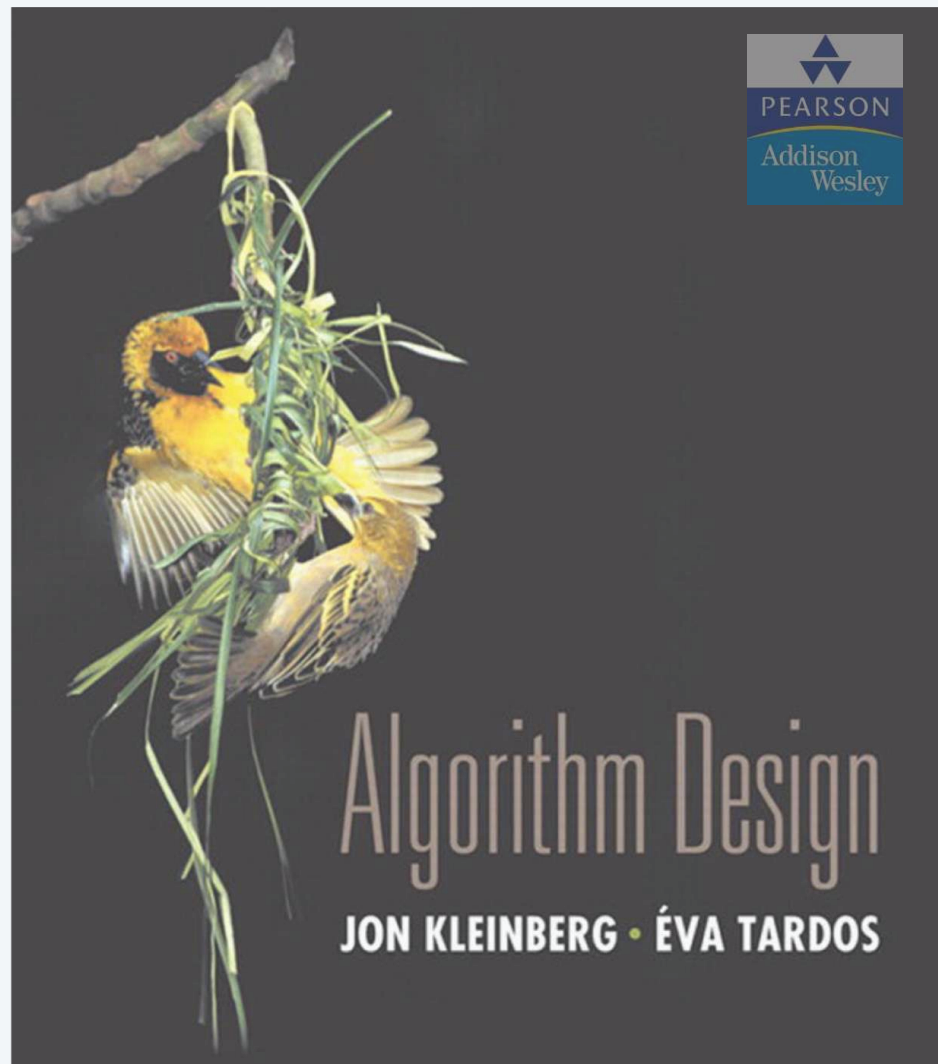
Operations research. Traveling salesperson problem.

Physics. Partition function of 3d Ising model.

Politics. Shapley–Shubik voting power.

Recreation. Versions of Sudoku, Checkers, Minesweeper, Tetris, Rubik's Cube.

Statistics. Optimal experimental design.



SECTION 8.9

8. INTRACTABILITY II

- *P vs. NP*
- *NP-complete*
- *co-NP*

Asymmetry of NP

Asymmetry of NP. We need short certificates only for *yes* instances.

Ex 1. SAT vs. UN-SAT.

- Can prove a CNF formula is satisfiable by specifying an assignment.
- How could we prove that a formula is not satisfiable?

SAT. Given a CNF formula Φ , is there a satisfying truth assignment?

UN-SAT. Given a CNF formula Φ , is there no satisfying truth assignment?

Asymmetry of NP

Asymmetry of NP. We need short certificates only for *yes* instances.

Ex 2. HAMILTON-CYCLE vs. NO-HAMILTON-CYCLE.

- Can prove a graph is Hamiltonian by specifying a permutation.
- How could we prove that a graph is not Hamiltonian?

HAMILTON-CYCLE. Given a graph $G = (V, E)$, is there a simple cycle Γ that contains every node in V ?

NO-HAMILTON-CYCLE. Given a graph $G = (V, E)$, is there no simple cycle Γ that contains every node in V ?

Asymmetry of NP

Asymmetry of NP. We need short certificates *only* for *yes* instances.

Q. How to classify UN-SAT and NO-HAMILTON-CYCLE ?

- SAT $\in$ **NP**-complete and SAT $\equiv_P$ UN-SAT.
- HAMILTON-CYCLE $\in$ **NP**-complete and HAMILTON-CYCLE $\equiv_P$ NO-HAMILTON-CYCLE.
- But neither UN-SAT nor NO-HAMILTON-CYCLE are known to be in **NP**.

NP and co-NP

NP. Decision problems for which there is a poly-time certifier.

Ex. SAT, HAMILTON-CYCLE, and COMPOSITES.

Def. Given a decision problem X , its **complement** $\bar{X}$ is the same problem with the *yes* and *no* answers reversed.

Ex. $X = \{ 4, 6, 8, 9, 10, 12, 14, 15, \dots \}$

$\bar{X} = \{ 2, 3, 5, 7, 11, 13, 17, 23, 29, \dots \}$

← ignore 0 and 1
(neither prime nor composite)

co-NP. Complements of decision problems in **NP**.

Ex. UN-SAT, NO-HAMILTON-CYCLE, and PRIMES.

NP = co-NP ?

Fundamental open question. Does **NP = co-NP**?

- Do *yes* instances have succinct certificates iff *no* instances do?
- Consensus opinion: no.

Theorem. If **NP $\neq$ co-NP**, then **P $\neq$ NP**.

Pf idea.

- **P** is closed under complementation.
- If **P = NP**, then **NP** is closed under complementation.
- In other words, **NP = co-NP**.
- This is the contrapositive of the theorem.